

Go assignment demonstrating safety in Go Routines and mutex

Assignment Objective:

- Simulate multiple users depositing money concurrently to one account.
- Show the race condition issue that occurs without synchronization.
- Understand why mutex is necessary for concurrent data access protection.

Scenario Setup:

- Bank account with a balance.
- Multiple goroutines represent users scanning a QR code and depositing money.
- Deposits happen concurrently, modifying the shared balance.
- Run the program without mutex and observe inconsistent results.

Step-by-step Assignment:

1. Define a struct for the account with a balance.
2. Create a function to deposit money modifying the balance.
3. Spin up multiple goroutines for concurrent deposits.
4. Run the program without mutex.
5. Observe inconsistent or incorrect final balance.
6. Discuss why this happens (race condition).

What to Observe:

- The expected balance is $\text{numUsers} * \text{depositAmount}$.
- The actual balance will often be less due to race conditions.
- The race condition is because multiple goroutines read and write balance concurrently.
- Fix using `sync.Mutex` to protect the deposit method.

Go Assignment: Emulating a Large Backup Task with Context and Worker Routine

Assignment Overview

You are tasked with emulating a long-running backup operation in Go using a combination of context and worker goroutines. This exercise will help you understand how to use Go's context package for graceful cancellation and timeout handling during the execution of concurrent tasks.

Objectives

- Simulate a resource-intensive backup job as a worker function running in its own goroutine.
- Use a context to allow for user-initiated cancellation of the backup operation.

- Demonstrate how the backup process can be canceled automatically after a set timeout period (e.g., simulating the user exiting after 3 seconds).
- Ensure the main program coordinates all tasks and handles cleanup appropriately upon cancellation.

Assignment Requirements

- 1. Simulate a Backup Task**
 - Design a worker function that emulates backing up a large number of files or data chunks.
 - The worker should report progress periodically (e.g., via printed output).
- 2. Implement Contextual Cancellation**
 - Use Go's context package to create a context, which can be canceled either by a user action (simulated timer) or completion of the backup.
 - The worker must periodically check the context; when cancellation is detected, it should halt processing and exit gracefully.
- 3. Simulate User Exit After 3 Seconds**
 - Set up a timer that triggers after 3 seconds, representing the user exiting the application.
 - When the timer fires, cancel the backup process using the context.
- 4. Graceful Shutdown and Reporting**
 - Ensure the main function waits for the worker routine to complete or be canceled.
 - Output a message indicating whether the backup completed successfully or was canceled.